

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский университет ИТМО»

ОТЧЁТ

по лабораторной работе

«Генераторы статических сайтов на Python: публикация результатов экспериментов»

Исследовательское задание T5 — публикуемость и цитируемость результата

Практическое задание P2 — стресс-тест научного контента

Выполнил: студент _____
(логин Helios s335989, GitHub ExcaliBBur)
Преподаватель: _____

Санкт-Петербург, 2026

Содержание

Ход работы	4
Ссылки	4
1–2. Python, pip, virtualenv	4
3. Каталог проекта и виртуальное окружение	4
4. Фиксация зависимостей и .gitignore	4
5. Установка генераторов и каркас сайтов	4
6. Локальная сборка	5
7. Репозиторий на GitHub	5
8. GitHub Actions и GitHub Pages	5
9. Учётные записи на отечественных площадках	6
10. Workflow для отечественных площадок	7
11. Базовый URL	7
12. Проверка результата развёртывания	8
13. Лицензии	8
14. Отладка	8
T5. Публикуемость и цитируемость результата	9
1. Постоянные ссылки и стабильность URL	9
2. Версионирование сайта и соответствие версий сайта, кода и данных	10
3. DOI и ORCID	10
4. Метаданные для индексации	11
5. Лицензирование	12
6. Соответствие принципам FAIR	12
7. Сводная матрица	12
8. Что достижимо при размещении на Helios	13
9. Реализация в проекте	14
10. Вывод по T5	14
P2. Страница — стресс-тест научного контента	15
1. Постановка	15
2. Обязательный состав страницы и способ реализации	15
3. Измерения	15
4. Что не удалось реализовать и какой ценой	23
5. Вывод по P2	24
Отладка	25
1. Формулы в MkDocs выводятся сырым TeX	25

2. mkdocs build --strict падает на ссылке на якорь	25
3. Снимет метки версии не раскрывается.....	25
4. Интерактивный график Plotly не растягивается на ширину колонки	26
5. Healthcheck падает с UnicodeEncodeError на Windows	26
6. Healthcheck не находит русское слово в индексе поиска MkDocs	26
7. scripts/measure.py не попадает в репозиторий	27
8. Внешние запросы у MkDocs при «полностью локальном» сайте	27
9. mkdocs-bibtex находит «цитату» внутри inline-кода.....	27
Вывод.....	28
Рекомендуемый стек	28
Условия, при которых рекомендация меняется	28
Приложение А. Тексты workflow.....	30
.github/workflows/pages.yml	30
.github/workflows/helios.yml.....	31
.github/workflows/vps.yml.....	32

Ход работы

Ссылки

Ресурс	Адрес
Репозиторий	https://github.com/ExcaliBBur/static_site
GitHub Pages — MkDocs + Material	https://excalibbur.github.io/static_site/
GitHub Pages — Sphinx + MyST	https://excalibbur.github.io/static_site/sphinx/
Helios ИТМО	https://se.ifmo.ru/~s335989/p2/
Собственный VPS (excalibbur.ru)	https://excalibbur.ru/static_site/
Запуски CI	https://github.com/ExcaliBBur/static_site/actions

1–2. Python, pip, virtualenv

Компонент	Версия	Команда проверки
Python	3.13.3	<code>python --version</code>
pip	25.0.1	<code>pip --version</code>
virtualenv	21.13.0	<code>python -m virtualenv --version</code>
ОС	Windows 11 Pro (локально), Ubuntu 24.04 (раннер CI)	—

virtualenv отсутствовал и был установлен командой `python -m pip install --user virtualenv`.

3. Каталог проекта и виртуальное окружение

```
python -m virtualenv .venv
.venv/Scripts/activate          # Linux/macOS: source .venv/bin/activate
```

4. Фиксация зависимостей и .gitignore

Выбран `requirements.txt` с точными версиями (`==`): для двух генераторов и скрипта подготовки контента это проще `lock`-файла `uv/poetry`, а на чистом раннере даёт ту же воспроизводимость в связке с точной версией Python (`python-version: "3.13.3"` в `workflow`).

Файл	Назначение
<code>requirements.txt</code>	всё, что нужно для сборки сайта (15 пакетов с точными версиями)
<code>requirements-dev.txt</code>	дополнительно: Playwright (замеры P2), python-docx (отчёт)
<code>.gitignore</code>	<code>.venv/</code> , <code>site/</code> , <code>_build/</code> , <code>public/</code> , <code>__pycache__</code> /, <code>.cache/</code> и все сгенерированные файлы (графики, копии отчёта и MathJax внутри сайтов)
<code>.gitattributes</code>	<code>eol=lf</code> : одинаковые окончания строк на Windows и в CI

5. Установка генераторов и каркас сайтов

```
pip install -r requirements.txt
```

Структура репозитория:

```
data/          набор данных эксперимента + метаданные (версия, sha256)
```

report/	текст отчёта – общий Markdown для обоих генераторов
scripts/prepare.py	данные → расчёт → графики, таблица, метка версии
scripts/build_all.py	prepare → mkdocs --strict → sphinx -W → public/
scripts/check_site.py	healthcheck опубликованного сайта
scripts/measure.py	замеры P2
scripts/build_docx.py	этот отчёт в формате Word
site-mkdocs/	MkDocs + Material (основной сайт, публикуется в корень)
site-sphinx/	Sphinx + MyST + Furo (публикуется в /sphinx/)
vendor/mathjax/	локальная копия MathJax 3.2.2
.github/workflows/	pages.yml, helios.yml, vps.yml

Отчёт хранится один раз в report/ и при каждой сборке копируется скриптом prepare.py в оба сайта, поэтому текст на сайтах MkDocs и Sphinx и в документе Word совпадает. Для этого в отчёте используется только общее подмножество Markdown (заголовки, таблицы, списки, код, изображения), которое одинаково понимают Python-Markdown и MyST.

6. Локальная сборка

```
python scripts/prepare.py
cd site-mkdocs && mkdocs serve          # предпросмотр с автообновлением
mkdocs build --strict                   # предупреждение = ошибка
cd ../site-sphinx && sphinx-build -W -b html . _build/html
python scripts/build_all.py             # всё вместе → public/
```

В mkdocs.yml включена проверка ссылок и якорей (validation.anchors: warn), поэтому --strict ловит и битые ссылки на разделы других страниц. Сработавшие на практике случаи описаны в разделе «Отладка» (ошибки 2 и 9).

7. Репозиторий на GitHub

```
git init -b main
git remote add origin https://github.com/ExcaliBBur/static_site.git
git push -u origin main
```

8. GitHub Actions и GitHub Pages

В настройках репозитория Pages → Source переключён на **GitHub Actions**. Workflow pages.yml состоит из двух job:

Job	Триггер	Что делает	Права
build	push в main, pull request, ручной запуск	установка зависимостей с кэшем pip → build_all.py (строгий режим) → healthcheck собранного сайта на локальном сервере → upload-pages-artifact	contents: read
deploy	только push в main	deploy-pages → healthcheck опубликованного URL с проверкой хеша коммита	pages: write, id-token: write

Сравнение двух подходов к публикации:

Критерий	push в ветку gh-pages (peaceiris/actions-gh-pages)	upload-pages-artifact + deploy-pages
Где хранится сайт	отдельная ветка с собранным HTML в самом репозитории	артефакт workflow (tar), ветки нет
Нужные права токена	contents: write — запись во	pages: write + id-token: write,

Критерий	push в ветку gh-pages (peaceiris/actions-gh-pages)	upload-pages-artifact + deploy-pages
	весь репозиторий	записи в репозиторий нет
Последствия компрометации шага	можно изменить код в любой ветке	можно только опубликовать другой сайт
Сторонний код в цепочке	action стороннего автора	только официальные actions GitHub
История версий сайта	есть (коммиты в gh-pages), откат — git revert	нет; откат — повторный запуск старого workflow
Размер репозитория	растёт с каждой публикацией	не меняется
Environment с правилами защиты	нет	да (github-pages)
Работает с другими хостингами	ветку можно зеркалировать куда угодно	только GitHub Pages

Выбрана официальная связка: меньше прав, нет стороннего кода, репозиторий не разрастается.

Все actions зафиксированы по SHA коммита с тегом в комментарии (actions/checkout@3d3c42e... # v7.0.1): тег можно перенести на другой коммит, SHA — нет.

9. Учётные записи на отечественных площадках

Площадка	Доступ	Адрес сайта
Helios ИТМО	учётная запись s335989, SSH se.ifmo.ru:2222, FreeBSD 14.5, nginx 1.30	~/public_html/p2/ → https://se.ifmo.ru/~s335989/p2/
Собственный VPS	excalibur.ru, Ubuntu 24.04, Nginx Proxy Manager в Docker	/root/npm/data/static_site/ → https://excalibur.ru/static_site/

Для каждой площадки создан **отдельный deploy-ключ ed25519** (не личный ключ разработчика):

Площадка	Строка в authorized_keys	Что может злоумышленник с этим ключом
Helios	restrict,command="/usr/local/sbin/rrsync /home/studs/s335989/public_html/p2/" ssh-ed25519 ...	только rsync внутри public_html/p2
VPS	restrict,command="/usr/bin/rrsync /root/npm/data/static_site/" ssh-ed25519 ...	только rsync внутри каталога сайта

restrict запрещает pty, форвардинг портов и агента, а принудительная команда rrsync пропускает только rsync и только в указанный каталог. Проверка — попытка выполнить команду с deploy-ключом:

```
$ ssh -i helios_deploy -p 2222 s335989@se.ifmo.ru 'ls /'
/usr/local/sbin/rrsync error: SSH_ORIGINAL_COMMAND does not run rsync
```

Первоначально ключ Helios был добавлен только с restrict (полный доступ к учётной записи студента) в предположении, что rrsync на FreeBSD-сервере нет; проверка which rrsync показала обратное, и ключ был ограничен так же, как на VPS.

Ключи хостов для known_hosts получены через ssh-keyscan и **сверены с отпечатками из уже доверенного локального ~/.ssh/known_hosts** (ED25519 SHA256:3n1x6Bq0... для Helios, SHA256:MH9WzKIF... для VPS). В CI known_hosts берётся из секрета, проверка ключа хоста не отключается (StrictHostKeyChecking=yes).

10. Workflow для отечественных площадок

helios.yml и vps.yml отличаются от pages.yml способом доставки: вместо артефакта Pages — rsync по SSH.

Секрет / переменная	Где используется	Содержимое
HELIOS_SSH_KEY	helios.yml	приватный deploy-ключ Helios
HELIOS_KNOWN_HOSTS	helios.yml	[se.ifmo.ru]:2222 ssh-ed25519 ...
HELIOS_USER (variable)	helios.yml	s335989
VPS_SSH_KEY	vps.yml	приватный deploy-ключ VPS
VPS_KNOWN_HOSTS	vps.yml	excalibbur.ru ssh-ed25519 ...

Ключевой шаг доставки:

```
rsync -rlz --checksum --delete-after --delay-updates --chmod=D755,F644 \  
-e "ssh -p 2222 -i ~/.ssh/helios -o StrictHostKeyChecking=yes -o IdentitiesOnly=yes" \  
public/ "s335989@se.ifmo.ru:./" # "./" = каталог, заданный rrsync на сервере
```

--delay-updates складывает новые файлы во временные имена и переименовывает их в конце, а --delete-after удаляет старые файлы только после передачи — при обрыве соединения на сервере остаётся прежняя целостная версия сайта, а не смесь двух.

На VPS для отдачи каталога добавлен файл /root/npm/data/nginx/custom/server_proxy.conf, который Nginx Proxy Manager штатно подключает в server хоста excalibbur.ru:

```
location = /static_site { return 301 /static_site/; }  
location ^~ /static_site/ {  
    alias /data/static_site/;  
    index index.html;  
    error_page 404 /static_site/404.html;  
    gzip on;  
    gzip_min_length 1024;  
    gzip_types text/css application/javascript application/json image/svg+xml text/xml;  
    add_header X-Content-Type-Options nosniff;  
}
```

Полные тексты workflow с комментариями приведены в приложении.

11. Базовый URL

Параметр	Значение	Зачем
site_url (MkDocs)	из переменной SITE_URL: свой для каждой площадки	канонический URL, sitemap.xml, пути в 404.html
html_baseurl (Sphinx)	SITE_URL + "sphinx/"	sitemap.xml, og:url
use_directory_urls	false	страницы вида report/t5.html: относительные ссылки из сырого HTML (iframe) работают одинаково из корня, из подкаталога /~s335989/p2/ и при открытии файла с диска

Типичная ошибка из задания воспроизведена и проверена: обычные страницы MkDocs ссылаются на ресурсы относительными путями и работают из любого подкаталога, но 404.html использует абсолютные пути от site_url:

Сборка	Ссылка на CSS в 404.html
SITE_URL=http://127.0.0.1:8000/	/assets/stylesheets/main.ec1eaa64.min.css
SITE_URL=https://se.ifmo.ru/~s000000/p2/	/~s000000/p2/assets/stylesheets/main.ec1eaa64.min.css

Поэтому каждая площадка собирается со своим SITE_URL (задан в соответствующем workflow), а не с адресом GitHub Pages.

12. Проверка результата развёртывания

Скрипт `scripts/check_site.py` запускается после каждого деплоя и завершает job с ошибкой при любом несоответствии:

Проверка	Как проверяется
HTTP 200	главная, страница P2 и T5 в MkDocs и Sphinx
Контрольная строка	p2-site-ok в HTML главной
Версия	метка сборки содержит хеш текущего коммита (ожидание до 3 минут для CDN GitHub Pages)
Поиск	search/search_index.json MkDocs содержит слово «градиентн...»; searchindex.js Sphinx доступен
Формулы без CDN	tex-svg.js и plotly.min.js отдаются с того же хоста; в <script src>/<link href> нет CDN-хостов
sitemap.xml	доступен у обоих генераторов

Результаты ручного запуска на отечественных площадках:

Площадка	Результат check_site.py	Время загрузки сайта (22 МБ, 136 файлов)	Сжатие	Своя страница 404
Helios (tar по SSH)	все проверки пройдены	43,6 с	нет (plotly.min.js 4,7 МиБ)	нет (страница nginx)
VPS (tar по SSH)	все проверки пройдены	50,9 с	gzip (plotly.min.js 1,7 МиБ)	да

Отображение формул при недоступных CDN дополнительно проверено в браузере: все внешние запросы блокировались, MathJax отрисовал все формулы (P2, § 3.4).

13. Лицензии

Объект	Лицензия	Файл
Текст, рисунки, таблицы	CC BY 4.0	LICENSE-CONTENT
Код	MIT	LICENSE
Данные	CC0 1.0	data/LICENSE
MathJax (вендорная копия)	Apache 2.0	vendor/mathjax/LICENSE

Сведения для цитирования — CITATION.cff. На сайтах есть страница «Лицензии» и машиночитаемая ссылка <link rel="license">.

14. Отладка

Все ошибки, возникшие при выполнении работы, описаны в разделе «Отладка».

T5. Публикуемость и цитируемость результата

Задача T5 — определить, что отличает «сайт с результатами» от полноценной публикации, и какие механизмы для этого дают генераторы из T1: **MkDocs + Material**, **Sphinx + MyST**, **Pelican** и **Quarto**.

Для каждого механизма используется трёхуровневая шкала:

Обозначение	Смысл
К — из коробки	работает штатными средствами генератора или его основной темы, достаточно настройки
П — плагин	нужен сторонний пакет/расширение, устанавливаемый из PyPI
Р — вручную	нужен собственный шаблон, статический файл, внешний сервис или организационное правило

Все утверждения вида «К/П/Р» для MkDocs и Sphinx проверены на практике в этом репозитории (см. раздел «Реализация в проекте» ниже); для Pelican и Quarto — по официальной документации.

1. Постоянные ссылки и стабильность URL

Постоянная ссылка — это URL, который не меняется при пересборке и реорганизации сайта. У всех рассмотренных генераторов URL страницы выводится из пути к исходному файлу, поэтому **пересборка сама по себе URL не меняет**: изменения происходят при переименовании файла, смене структуры каталогов или переключении режима формирования адресов.

Генератор	Откуда берётся URL	Типичная ловушка	Редиректы со старых адресов
MkDocs	путь к .md; use_directory_urls превращает a/b.md в a/b/ или a/b.html	переключение use_directory_urls меняет все адреса сайта	П: mkdocs-redirects (HTML-страницы с meta refresh)
Sphinx	имя документа (docname) + .html; dirhtml-билдер даёт a/b/	смена билдера html ↔ dirhtml	П: sphinx-reredirects
Pelican	шаблоны ARTICLE_URL/ARTICLE_SAVE_AS и метаданное slug	slug по умолчанию выводится из заголовка — правка заголовка меняет URL	Р (или плагин)
Quarto	путь к .qmd	переименование файла	К: поле aliases: в заголовке документа

Якоря внутри страницы менее стабильны, чем адреса страниц: они выводятся из текста заголовков, и при правке заголовка ссылка вида page.html#раздел начинает вести на начало страницы. Поэтому для цитируемых мест (таблиц, рисунков, формул) следует задавать **явные идентификаторы**: в Sphinx это метки (label)= и :name:, в MkDocs — атрибуты {#id} и attrs: {id: ...}. В этом проекте все объекты, на которые ссылается текст, имеют явные id (tab-results, fig-static, eq:mse/eq-mse, lst-gd).

Режим `mkdocs build --strict с validation.anchors: warn` проверяет существование якорей при каждой сборке, т. е. «битая» внутренняя ссылка не попадёт в публикацию. В Sphinx ту же роль выполняет флаг `-W` и встроенный `linkcheck` для внешних ссылок.

Вывод. Стабильность URL — прежде всего организационная политика (не переименовывать опубликованные страницы, фиксировать режим URL), генератор лишь помогает её соблюдать: проверкой ссылок при сборке и редиректами.

2. Версионирование сайта и соответствие версий сайта, кода и данных

Опубликованный результат должен однозначно отвечать на вопрос «из какого кода и каких данных получено это число». Для этого нужны две вещи:

1. **Несколько версий сайта одновременно** (/v1.0/, /v1.1/, /latest/), чтобы старая ссылка продолжала показывать старые результаты.
2. **Метка происхождения на каждой странице:** хеш коммита, дата сборки, версия набора данных.

Механизм	MkDocs + Material	Sphinx	Pelican	Quarto
Несколько версий, переключатель	П: mike + К: селектор версий в Material (extra.version.provider: mike)	П: sphinx-multiversion (или хостинг Read the Docs)	Р: отдельные каталоги	Р: отдельные каталоги / профили проекта
Номер версии в шаблоне	Р: extra в mkdocs.yml	К: version/release в conf.py	Р	К: version в метаданных
Дата последнего изменения страницы	П: git-revision-date-localized	К: html_last_updated_fmt; П: sphinx-last-updated-by-git	К: modified в метаданных	К: date-modified
Хеш коммита / версия данных	Р	Р	Р	Р

В проекте метка происхождения формируется скриптом scripts/prepare.py и выводится на главной странице и странице стресс-теста обоих сайтов: Коммит <sha> · сборка <дата UTC> · набор данных v1.0 (sha256 ...). Хеш набора данных хранится в data/dataset.meta.json и пересчитывается при изменении dataset.csv, поэтому расхождение данных и опубликованного графика видно сразу.

3. DOI и ORCID

DOI не создаётся генератором. Правильная схема — архивировать зафиксированную версию репозитория в Zenodo:

3. Репозиторий подключается к Zenodo (GitHub → Zenodo integration).
4. Создаётся git-тег и GitHub Release (v1.0.0).
5. Zenodo по вебхуку забирает архив релиза, присваивает **DOI версии** и общий **concept DOI**, всегда указывающий на последнюю версию.
6. Метаданные записи (авторы, ORCID, лицензия, ключевые слова) Zenodo берёт из .zenodo.json или из CITATION.cff в корне репозитория.

Файл CITATION.cff в этом репозитории одновременно включает кнопку «Cite this repository» на GitHub и служит источником метаданных для Zenodo. DOI должен указывать на **лендинг Zenodo**, а не на сайт: у Zenodo есть гарантия хранения, а адрес учебного хостинга может исчезнуть.

ORCID — идентификатор автора, который снимает неоднозначность имени. Он указывается в CITATION.cff (поле orcid), в мета-теге citation_author_orcid и в видимой подписи автора.

Механизм	MkDocs + Material	Sphinx	Pelican	Quarto
----------	-------------------	--------	---------	--------

Механизм	MkDocs + Material	Sphinx	Pelican	Quarto
DOI версии	P (Zenodo)	P (Zenodo)	P (Zenodo)	P (Zenodo); K: поле doi: выводится в блоке цитирования
Блок «Как цитировать» с BibTeX	P	P	P	K: citation: true генерирует блок и BibTeX
ORCID автора	P: шаблон	P: шаблон	P: шаблон	K: author: - orcid:

4. Метаданные для индексации

Механизм	Назначение	MkDocs + Material	Sphinx	Pelican	Quarto
Highwire Press (citation_*)	индексация Google Scholar	P: override extrahead	P: шаблон page.html	P: шаблон	K: google-scholar: true
Dublin Core (dc:*)	библиотечные каталоги, репозитории	P: override	P: шаблон	P: шаблон	P: include-in-header
Open Graph (og:*)	превью в соцсетях и мессенджерах	K: плагин social темы Material (генерирует и карточки)	П: sphinxext-opengraph	П: плагин из pelican-plugins	K: open-graph: true
<meta name="description">	сниппет поисковика	K: site_description, description в front matter	K: директива meta	K: summary	K: description
sitemap.xml	обход сайта роботами	K: формируется всегда, нужен site_url	П: sphinx-sitemap	П: pelican-sitemap	K: при заданном site-url
robots.txt	правила обхода	P: статический файл	P: html_extra_path	P: STATIC_PATHS	P: resources
Канонический URL (rel=canonical)	борьба с дублями (GitHub Pages + Helios)	K: из site_url	K: html_baseurl	K: SITEURL	K: site-url

Особенность, проверенная на практике: **канонический URL и sitemap зависят от базового адреса**. Один и тот же сайт публикуется на GitHub Pages и на Helios, поэтому site_url (MkDocs) и html_baseurl (Sphinx) задаются переменными окружения в workflow, и каждое зеркало собирается со своим адресом. Проверка показала, что это обязательно: обычные страницы MkDocs ссылаются на ресурсы относительными путями и работают из любого каталога, но 404.html, rel=canonical и sitemap.xml строятся от site_url. При сборке зеркала для Helios с адресом GitHub Pages страница 404 запрашивала бы стили по пути /static_site/assets/..., которого на Helios нет. Побочный эффект — у двух зеркал разные канонические адреса, и поисковик может счесть их дублями. Правильное решение — отдельная переменная для rel=canonical (всегда адрес основного сайта) при site_url, равном фактическому адресу зеркала; в проекте оно не реализовано, основной адрес — GitHub Pages.

Пример фактически сгенерированных тегов (MkDocs, страница стресс-теста):

```
<meta name="citation_title" content="Сравнение генераторов статических сайтов ...">
<meta name="citation_author" content="ExcaliBBur">
<meta name="citation_author_orcid" content="https://orcid.org/0000-0000-0000-0000">
<meta name="citation_publication_date" content="2026/09/29">
<link rel="schema.DC" href="http://purl.org/dc/elements/1.1/">
<meta name="DC.title" content="Стресс-тест научного контента ... — Публикация результатов экспериментов">
<meta name="DC.rights" content="https://creativecommons.org/licenses/by/4.0/">
<meta property="og:type" content="article">
<link rel="license" href="https://creativecommons.org/licenses/by/4.0/">
```

Ограничение Highwire Press: Google Scholar индексирует прежде всего страницы, похожие на статьи (заголовок, авторы, дата, PDF по citation_pdf_url), и значительно

лучше — на доменах научных организаций и репозиториях. Мета-теги необходимы, но не достаточны; для реальной индексации надёжнее Zenodo.

5. Лицензирование

Текст, код и данные — разные объекты авторского права, для них подходят разные лицензии:

Объект	Лицензия в проекте	Почему
Текст, рисунки, таблицы	CC BY 4.0 (LICENSE-CONTENT)	стандарт для научных публикаций, требует только указания авторства
Код скриптов, конфигурация, workflow	MIT (LICENSE)	СС-лицензии не предназначены для ПО (нет положений о патентах и об отказе от гарантий на код)
Данные	CC0 1.0 (data/LICENSE)	синтетические данные; CC0 снимает вопрос о «цепочке атрибуции» при агрегации
Сторонние файлы	исходные лицензии (MathJax — Apache 2.0, Plotly.js — MIT)	обязаны сохраняться при распространении

Ни один генератор не управляет лицензиями «из коробки», кроме Quarto (поле `license`: выводит лицензию в приложении к документу). В MkDocs и Sphinx лицензия выводится через `copyright` в подвале (K), а машиночитаемая ссылка `<link rel="license">` добавлена шаблоном (P).

6. Соответствие принципам FAIR

Принципы FAIR [Wilkinson et al., 2016] сформулированы для данных, но применимы к любому исследовательскому результату.

Принцип	Требование	Чем обеспечено в проекте	Роль генератора
F — Findable	постоянный глобальный идентификатор, богатые метаданные, индексированность	DOI через Zenodo (план), CITATION.cff, Highwire/DC/OG-теги, sitemap.xml	частичная: sitemap, meta, стабильные URL
A — Accessible	доступ по стандартному протоколу, метаданные доступны даже без данных	HTTPS на GitHub Pages; архив в Zenodo переживёт сайт	минимальная: генератор лишь создаёт статический HTML
I — Interoperable	открытые форматы, стандартные словари	CSV + JSON-метаданные, BibTeX, Dublin Core, CFF	частичная: форматы вывода
R — Reusable	лицензия, происхождение (provenance), описание метода	раздельные лицензии, метка коммита и версии данных, скрипт <code>prepare.py</code>	частичная: вывод метки версии

FAIR — свойство всей системы публикации (репозиторий + архив + сайт), а не одного генератора.

7. Сводная матрица

Механизм	MkDocs + Material	Sphinx + MyST	Pelican	Quarto
Стабильные URL, проверка внутренних ссылок	K	K	K	K
Редиректы со старых URL	П	П	P	K
Версии сайта с переключателем	П (mike)	П (sphinx-multiversion)	P	P

Механизм	MkDocs + Material	Sphinx + MyST	Pelican	Quarto
Метка коммита/данных на странице	P	P	P	P
DOI	P (Zenodo)	P (Zenodo)	P (Zenodo)	P + K (вывод поля doi)
ORCID	P	P	P	K
Блок «Как цитировать»	P	P	P	K
Highwire Press	P	P	P	K
Dublin Core	P	P	P	P
Open Graph	K (плагин темы)	П	П	K
sitemap.xml	K	П	П	K
robots.txt	P	P	P	P
Раздельные лицензии	P	P	P	K (одна лицензия) + P
FAIR целиком	P	P	P	P

Quarto ближе всех к «публикации из коробки» (цитирование, ORCID, Highwire, DOI в метаданных), но это не Python-пакет, а отдельный бинарный дистрибутив. MkDocs + Material и Sphinx покрывают недостающее плагинами и переопределением одного блока шаблона — в проекте это заняло около 30 строк Jinja2 на генератор.

8. Что достижимо при размещении на Helios

Helios — учебный сервер ИТМО (FreeBSD 14). Файлы из ~/public_html отдаёт nginx 1.30 по адресу <https://se.ifmo.ru/~<логин>/>; сайт работы размещён в <https://se.ifmo.ru/~s335989/p2/>. Для сравнения тот же сайт выложен на собственный VPS (Ubuntu 24.04, Nginx Proxy Manager): https://excalibur.ru/static_site/. Свойства хостингов проверены запросами curl к опубликованным сайтам.

Механизм	Helios	Собственный VPS	Обоснование
Статический HTML, CSS, JS, поиск	да	да	поиск работает на клиенте по search_index.json / searchindex.js
Формулы и графики без интернета	да	да	MathJax и Plotly.js лежат в самом сайте
Мета-теги Highwire, DC, OG	да	да	это часть HTML, хостинг не влияет
Кодировка UTF-8 в заголовке ответа	да (text/html; charset=UTF-8)	да	для листинга каталога Helios отдаёт ISO-8859-1, для .html — UTF-8
Сжатие gzip	нет: plotly.min.js передаётся целиком, 4,7 МиБ	да: 1,7 МиБ	на Helios сжатие не настроено, и пользователь не может его включить
Собственная страница 404	нет: отдаётся страница nginx	да: error_page 404	конфигурация nginx Helios недоступна пользователю
Серверные редиректы 301	нет	да	nginx не читает .htaccess; на Helios остаются только HTML-страницы с meta refresh
sitemap.xml	частично	да	на Helios файл публикуется, но сослаться на него из robots.txt нельзя; регистрировать вручную в Search Console
robots.txt	нет	да	роботы читают только /robots.txt в корне домена se.ifmo.ru
Постоянный URL	нет	пока оплачены домен и сервер	адрес на Helios привязан к учётной записи студента
Собственный домен, HTTPS	нет; HTTPS домена se.ifmo.ru	да; сертификат Let's Encrypt	виртуальные хосты на Helios пользователям не предоставляются

Механизм	Helios	Собственный VPS	Обоснование
Версии /v1.0/, /latest/	да	да	это просто подкаталоги; versions.json от mike работает на любом статическом хостинге
DOI, указывающий на сайт	не рекомендуется	не рекомендуется	DOI должен вести на долговременный ресурс — лендинг Zenodo
Индексация Google Scholar	маловероятна	маловероятна	персональные сайты индексируются хуже, чем Zenodo

Итог: Helios подходит как **зеркало** и площадка для демонстрации в отечественной инфраструктуре, но не как место «канонической» публикации. Собственный VPS снимает технические ограничения (сжатие, 404, редиректы, robots.txt, домен), но постоянство адреса по-прежнему зависит от оплаты сервера. Канонический адрес — GitHub Pages (или собственный домен), долговременное хранение и DOI — Zenodo.

9. Реализация в проекте

Механизм	MkDocs	Sphinx
Highwire + Dublin Core	site-mkdocs/overrides/main.html (блок extrahead)	site-sphinx/_templates/page.html
Open Graph	тот же override	sphinxext-opengraph
sitemap.xml	встроенный	sphinx-sitemap
robots.txt	site-mkdocs/docs/robots.txt	site-sphinx/_extra/robots.txt
Канонический URL	site_url: !ENV SITE_URL	html_baseurl из SPHINX_BASEURL
Метка версии	scripts/prepare.py → snippets/build_info.md	scripts/prepare.py → _generated/build_info.md
Лицензии	LICENSE, LICENSE-CONTENT, data/LICENSE, страница «Лицензии»	та же страница
Цитирование	CITATION.cff в корне репозитория	—

10. Вывод по T5

Сайт с результатами становится публикацией, когда выполнены три условия: результат **идентифицируем** (DOI и ORCID), **неизменяем для процитированной версии** (версии сайта + метка коммита и данных) и **машиночитаем** (Highwire/DC/OG, sitemap, CFF, отдельные лицензии). Генератор обеспечивает только третье условие и частично второе; первое — задача внешней инфраструктуры (Zenodo, ORCID). Из рассмотренных генераторов больше всего «из коробки» даёт Quarto, из Python-генераторов — MkDocs + Material, где недостающее добавляется одним override-шаблоном и плагином mike.

Р2. Страница — стресс-тест научного контента

1. Постановка

Одна и та же страница сверстана на двух генераторах из T1:

Генератор	Версии	Тема и расширения
MkDocs + Material	MkDocs 1.6.1, Material 9.7.7	pymdownx 12.1 (arithmatex, highlight, snippets, blocks.caption), mkdocs-bibtex 4.4.0
Sphinx + MyST	Sphinx 9.1.0, MyST-Parser 5.1.0	тема Furo, sphinxcontrib-bibtex 2.7.0, sphinx-design 0.7.0, sphinx-sitemap, sphinxext-opengraph

Содержимое страницы — результаты небольшого эксперимента: линейная регрессия, обученная градиентным спуском с тремя скоростями обучения, и аналитическое решение МНК. Числа, графики и таблица не вписаны вручную, а формируются скриптом `scripts/prepare.py` из `data/dataset.csv` при каждой сборке. Скрипт выдаёт один и тот же результат в двух форматах, которые понимает каждый генератор (HTML-таблица для MkDocs, grid table reStructuredText для Sphinx).

Страницы: MkDocs · Sphinx — `sphinx/p2/stress-test.html` на том же сайте.

2. Обязательный состав страницы и способ реализации

Элемент	MkDocs + Material	Sphinx + MyST
Нумерованная формула + ссылка из текста	<code>\begin{equation}...\label{eq:mse} + \eqref{eq:mse}</code> ; нумерует MathJax (tags: "ams") в браузере	директива <code>{math}</code> с <code>:label:</code> + роль <code>{eq}</code> ; номер присваивает Sphinx при сборке
Таблица с объединёнными ячейками и подписью	сырой HTML (<code>rowspan/colspan</code>), подпись — <code>/// table-caption</code> с автонумерацией	grid table rST внутри <code>{eval-rst}</code> , директива <code>table</code> с <code>:name:</code> , ссылка <code>{numref}</code>
Статический график (Matplotlib)	Markdown-изображение + <code>/// figure-caption</code>	директива <code>{figure}</code> с <code>:name:</code>
Интерактивный график (Plotly)	<code><iframe></code> на HTML, сгенерированный Plotly; <code>plotly.min.js</code> лежит рядом	то же через <code>{raw} html</code>
Листинг с подсветкой и номерами строк	<code>linenums="1" + title= (pymdownx.highlight)</code>	<code>{code-block}</code> с <code>:linenos:</code> , <code>:caption:</code> , <code>:emphasize-lines:</code>
Цитирование из .bib + список литературы	плагин <code>mkdocs-bibtex</code> : ключ с «собакой» в квадратных скобках → сноска, маркер <code>\bibliography</code>	<code>sphinxcontrib-bibtex: {cite:p}</code> → «[Автор, год]», директива <code>{bibliography}</code>
Двухколоночный блок	<code><div class="grid two-col" markdown></code> + 6 строк CSS	<code>::::{grid} 1 1 2 2</code> (sphinx-design)
Сноска	<code>[^1r]</code> (расширение footnotes)	<code>[^1r]</code> (встроено в MyST)
Перекрёстная ссылка на другой раздел	<code>[текст](../report/t5.md#якорь)</code> , существование якоря проверяет <code>-strict</code>	та же Markdown-ссылка (<code>myst_heading_anchors = 3</code>) или <code>{ref}</code>

3. Измерения

Условия: Windows 11, Python 3.13.3, Chrome (Playwright 1.63, канал chrome), локальный сервер `http.server` без сжатия. Скрипт — `scripts/measure.py`, сырые данные — `report/measurements.json`.

3.1. Время сборки и объём результата

Холодная сборка — каталог результата удалён; инкрементальная — повторный запуск без изменений исходников. По 3 запуска, время подготовки контента (`prepare.py`, $\approx 1,0$ с, общее для обоих) не включено.

Показатель	MkDocs + Material	Sphinx + MyST
Холодная сборка, запуски, с	1,47 / 0,89 / 0,92	2,11 / 2,00 / 1,99
Холодная сборка, среднее, с	1,09	2,04
Инкрементальная сборка, запуски, с	0,92 / 0,93 / 0,94	2,52 / 1,53 / 1,51
Инкрементальная сборка, среднее, с	0,93	1,85
Объём каталога результата, МиБ	11,65	7,59
Число файлов	67	64

MkDocs не имеет настоящей инкрементальной сборки (каждый раз пересобираются все страницы), но на 8 страницах это быстрее, чем запуск Sphinx с чтением окружения. Первая холодная сборка MkDocs медленнее из-за прогрева кэша файловой системы и импорта модулей. Каталог MkDocs больше за счёт файлов поддержки поиска на 30+ языках (`search/lunr.*.min.js`) и ресурсов темы.

3.2. Вес страницы и сетевые запросы

Учтены все ресурсы, загруженные при открытии страницы, включая `iframe` с Plotly. «Внешние» — запросы к хостам, отличным от хоста сайта.

Показатель	MkDocs + Material	Sphinx + MyST
Запросов всего	18	16
Внешних запросов	2 (<code>api.github.com</code>)	0
Вес страницы (без сжатия), КиБ	7 236	7 031
— HTML	51	47
— CSS	150	123
— JavaScript	6 878	6 792
— изображения	69	69
— индекс поиска и прочее	88	0 (грузится только на странице поиска)
HTML страницы, gzip, КиБ	8,9	8,2

Почти весь вес — два JavaScript-файла, одинаковых для обоих генераторов:

Ресурс	Без сжатия, КиБ	gzip, КиБ	Доля веса страницы
<code>plotly.min.js</code> (Plotly.js, полная сборка)	4 707	1 438	$\approx 65\%$
<code>tex-svg.js</code> (MathJax 3.2.2, $\text{TeX} \rightarrow \text{SVG}$)	2 059	662	$\approx 29\%$
Всё остальное (HTML, CSS, тема, PNG)	$\approx 470 / 265$	—	$\approx 6\%$

Вывод: стоимость интерактивного графика Plotly — около 4,7 МиБ (1,4 МиБ **gzip**), это на порядок больше самой страницы. Статический PNG того же графика — 69 КиБ. Для страницы с одним графиком разумнее частичная сборка `plotly.js-basic-dist-min` (≈ 1 МиБ) или загрузка `iframe` по клику.

Два внешних запроса MkDocs — это обращение темы Material к GitHub API за числом звёзд и форков репозитория (включается параметром `repo_url`). Запрос необязательный: при блокировке страница отображается полностью (см. 3.4).

3.3. Lighthouse

Оценка получена через Google PageSpeed Insights API (Lighthouse, мобильный профиль) для опубликованной на GitHub Pages версии.

3.4. Работа без внешних CDN и на мобильном разрешении

Проверка в Chrome: в режиме «без CDN» все запросы к хостам, кроме хоста сайта, принудительно обрываются; мобильный режим — 390×844, DPR 2, touch.

Проверка	MkDocs + Material	Sphinx + MyST
Формул отрисовано MathJax / осталось сырого TeX (без CDN)	15 / 0	8 / 0
График Plotly построен без CDN	да	да
Заблокировано запросов в режиме без CDN	2 (GitHub API)	0
Горизонтальная прокрутка на 390 px	нет (0 px)	нет (0 px)
Колонки на 1280 px / на 390 px	2 / 1	2 / 1
Широкая таблица на 390 px	прокручивается внутри блока	прокручивается внутри блока

Число «формул» различается из-за разметки: MathJax в MkDocs создаёт отдельный контейнер и для каждой ссылки `\eqref`, а Sphinx рендерит номера и ссылки без MathJax.

3.5. Скриншоты

Публикация результатов экспериментов

Главная

P2. Стресс-тест

Отчёт

Ход работы

T5. Публикуемость и цитируемость

P2. Сравнение генераторов

Отладка

Вывод

Лицензии

Стресс-тест научного контента (MkDocs + Material)

Коммит 96dbd18 · сборка 2026-09-29 16:13 UTC · набор данных v1.0 (sha256 d4b31c5c5254)

Страница публикует результаты небольшого вычислительного эксперимента: обучение линейной регрессии градиентным спуском с разными скоростями обучения и сравнение с аналитическим решением методом наименьших квадратов (МНК). Графики и таблица формируются скриптом `scripts/prepare.py` из файла `data/dataset.csv` на этапе сборки, т. е. числа на странице всегда соответствуют текущей версии данных и кода ².

Постановка эксперимента

Качество модели $\hat{y} = wx + b$ оценивается среднеквадратичной ошибкой:

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2 \quad (1)$$

Параметры обновляются по правилу градиентного спуска со скоростью обучения η :

$$w \leftarrow w - \eta \frac{\partial \text{MSE}}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial \text{MSE}}{\partial b}. \quad (2)$$

Минимизируемый функционал — формула (1), шаг обновления — формула (2).
Подробное обоснование выбора метрики приведено в учебнике ³.

Результаты

Таблица 1. Итоговая ошибка после 60 эпох (объединённые ячейки: `rowspan`, `colspan`)

Содержание

Постановка эксперимента

Результаты

Реализация

Интерпретация

Литература

Метод	η	Итоговая MSE		Эпох до сходимости
		обучающая	контрольная	
Градиентный спуск	0,0100	0,4516	0,4701	> 60
	0,0500	0,2616	0,2611	> 60
	0,2000	0,2488	0,2499	39
МНК (аналитически)	—	0,2487	0,2502	—

Кривые обучения на контрольной выборке показаны на рисунке 1; график построен библиотекой Matplotlib ⁴.

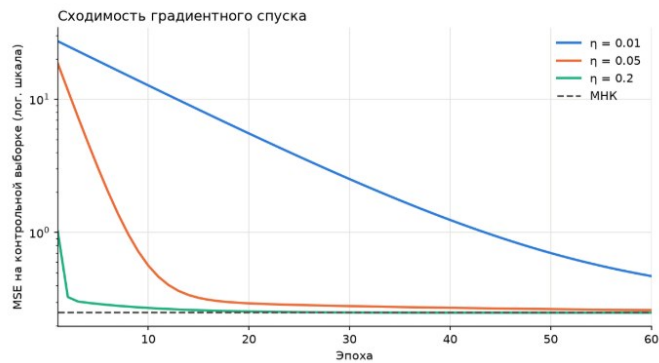


Рисунок 1. Сходимость градиентного спуска (статический PNG, Matplotlib)

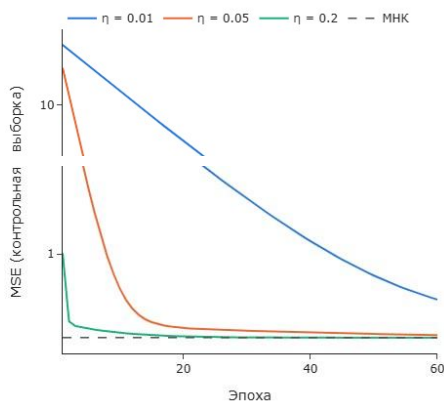


Рисунок 2. Тот же результат в Plotly: наведение показывает значения всех серий на эпохе

Реализация

Ядро эксперимента — функция одного прогона градиентного спуска (листинг 1).

Листинг 1. Градиентный спуск для парной линейной регрессии

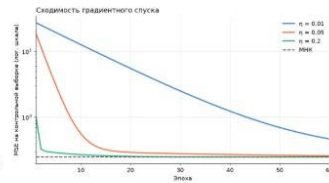
```
scripts/prepare.py

1 def gradient_descent(x, y, xv, yv, lr):
2     w = b = 0.0
3     train, val = [], []
4     for _ in range(EPOCHS):
5         err = w * x + b - y
6         w -= lr * 2 * np.mean(err * x)
7         b -= lr * 2 * np.mean(err)
8         train.append(mse(y, w * x + b))
9         val.append(mse(yv, w * xv + b))
10    return train, val
```

Строки 6–7 реализуют формулу (2).

Интерпретация

При $\eta = 0,2$ градиентный спуск за 39 эпох достигает ошибки аналитического решения (см. таблицу 1), тогда как при $\eta = 0,01$ за 60 эпох ошибка остаётся почти вдвое выше. Малый шаг гарантирует устойчивость, но требует больше итераций¹. Вопросы публикации и цитирования таких результатов разобраны в разделе «Постоянные ссылки» отчёта по T5, а принципы FAIR — в работе⁵.



Литература

1. При $\eta > 1/\lambda_{\max}$, где $\lambda_{\max}$ — наибольшее собственное значение гессиана, итерации расходятся.
2. Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi:10.1126/science.1213847.
3. Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
4. John D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi:10.1109/MCSE.2007.55.
5. Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, and others. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi:10.1038/sdata.2016.18.

Назад
Главная

Вперед
Ход работы

Текст — CC BY 4.0, код — MIT
Made with Material for MkDocs

MkDocs + Material, настольное разрешение 1280 px (страница целиком).

Публикация результатов экспериментов

🔍 Поиск

p2

Стресс-тест научного контента (Sphinx + MyST)

отчёт

Ход работы

T5. Публикуемость и цитируемость результата

p2

debugging

conclusion

Лицензии

Стресс-тест научного контента (Sphinx + MyST)

Коммит 96fbd18 · сборка 2026-09-29 16:13 UTC · набор данных v1.0 (sha256 d4b31c5c5254)

Страница публикует результаты небольшого вычислительного эксперимента: обучение линейной регрессии градиентным спуском с разными скоростями обучения и сравнение с аналитическим решением методом наименьших квадратов (МНК). Графики и таблица формируются скриптом scripts/prepare.py из файла data/dataset.csv на этапе сборки, т. е. числа на странице всегда соответствуют текущей версии данных и кода [Peng, 2011].

Постановка эксперимента

Качество модели $\hat{y} = wx + b$ оценивается среднеквадратичной ошибкой:

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2 \quad (1)$$

Параметры обновляются по правилу градиентного спуска со скоростью обучения η :

$$w \leftarrow w - \eta \frac{\partial \text{MSE}}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial \text{MSE}}{\partial b}. \quad (2)$$

Минимизируемый функционал — формула (1), шаг обновления — формула (2). Подробное обоснование выбора метрики приведено в учебнике [Goodfellow et al., 2016].

Результаты

Таблица 1 Итоговая ошибка после 60 эпох (объединённые ячейки в grid table)

Метод	η	Итоговая MSE		Эпох до сходимости
		обучающая	контрольная	
Градиентный спуск	0,0100	0,4516	0,4701	> 60
	0,0500	0,2616	0,2611	> 60
	0,2000	0,2488	0,2499	39
МНК (аналитически)	—	0.2487	0.2502	—

19

Кривые обучения на контрольной выборке показаны на [Рисунок 1](#); график построен библиотекой Matplotlib [[Hunter, 2007](#)].

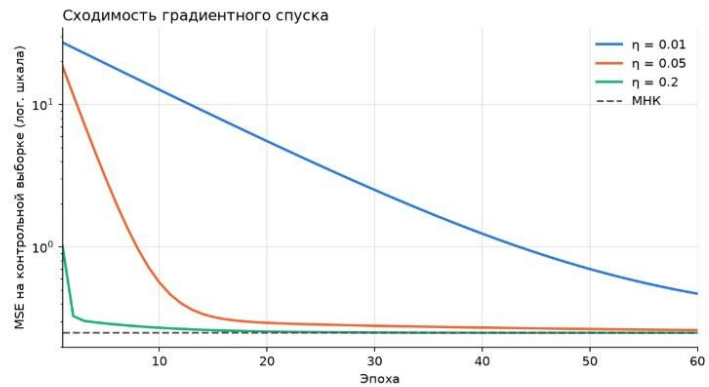
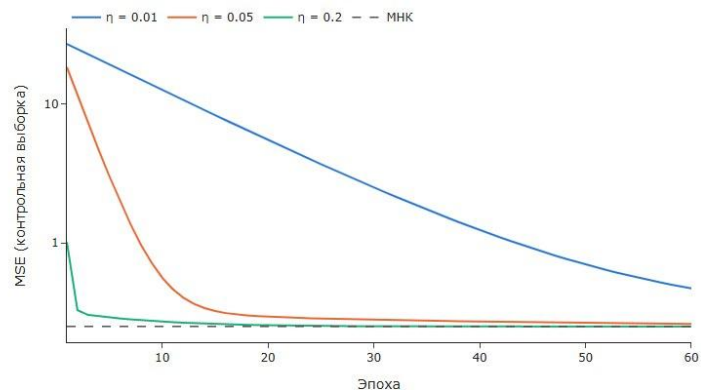


Рисунок 1 Сходимость градиентного спуска (статический PNG, Matplotlib)



Тот же результат в Plotly: наведение показывает значения всех серий на эпохе (подпись не нумеруется: директива `figure` в Sphinx принимает только изображение).

Реализация

Ядро эксперимента — функция одного прогона градиентного спуска ([Листинг 1](#)).

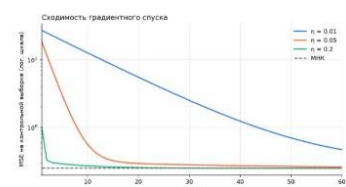
Листинг 1: градиентный спуск для парной линейной регрессии (`scripts/prepare.py`)

```
1 def gradient_descent(x, y, xv, yv, lr):
2     w = b = 0.0
3     train, val = [], []
4     for _ in range(EPOCHS):
5         err = w * x + b - y
6         w -= lr * 2 * np.mean(err * x)
7         b -= lr * 2 * np.mean(err)
8         train.append(mse(y, w * x + b))
9         val.append(mse(yv, w * xv + b))
10    return train, val
```

Строки 6–7 реализуют формулу (2).

Интерпретация

При $\eta = 0,2$ градиентный спуск за 39 эпох достигает ошибки аналитического решения (см. [Таблица 1](#)), тогда как при $\eta = 0,01$ за 60 эпох ошибка остаётся почти вдвое выше. Малый шаг гарантирует устойчивость, но требует больше итераций^[1]. Вопросы публикации и цитирования таких результатов разобраны в разделе «[Постоянные ссылки](#)» отчёта по T5, а принципы FAIR — в работе [[Wilkinson et al., 2016](#)].



Литература

- [1] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi:10.1126/science.1213847.

- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [3] John D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi:10.1109/MCSE.2007.55.
- [4] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, and others. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi:10.1038/sdata.2016.18.
- [1] При $\eta > 1/\lambda_{\max}$, где $\lambda_{\max}$ — наибольшее собственное значение гессиана, итерации расходятся.

< Previous
Home

Next >
Ход работы

Copyright © 2026, ExcaliB8ur. Текст — CC BY 4.0, код — MIT
Made with Sphinx and @pradyunsg's Furo
Last updated on 2026-09-29

Sphinx + MyST (Furo), настольное разрешение 1280 px.

≡ Публикация результа... 🔍

Стресс-тест научного контента (MkDocs + Material)

Коммит 96dbd18 · сборка 2026-09-29 16:13 UTC · набор данных v1.0 (sha256 d4b31c5c5254)

Страница публикует результаты небольшого вычислительного эксперимента: обучение линейной регрессии градиентным спуском с разными скоростями обучения и сравнение с аналитическим решением методом наименьших квадратов (МНК). Графики и таблица формируются скриптом `scripts/prepare.py` из файла `data/dataset.csv` на этапе сборки, т. е. числа на странице всегда соответствуют текущей версии данных и кода ².

Постановка эксперимента

Качество модели $\hat{y} = wx + b$ оценивается среднеквадратичной ошибкой:

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2 \quad (1)$$

Параметры обновляются по правилу градиентного спуска со скоростью обучения η :

$$w \leftarrow w - \eta \frac{\partial \text{MSE}}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial \text{MSE}}{\partial b}$$

Минимизируемый функционал — формула (1), шаг обновления — формула (2). Подробное обоснование выбора метрики приведено в учебнике ³.

Кривые обучения на контрольной выборке показаны на рисунке 1; график построен библиотекой Matplotlib ⁴.

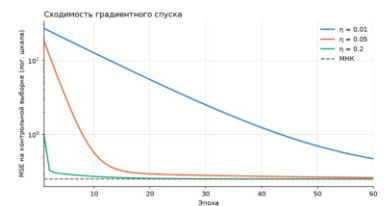
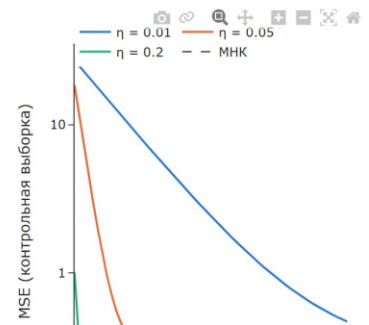


Рисунок 1. Сходимость градиентного спуска (статический PNG, Matplotlib)

Результаты

Таблица 1. Итоговая ошибка после 60 эпох (объединённые ячейки: `rowspan`, `colspan`)

Метод	η	Итоговая MSE	
		обучающая	контрольная
Градиентный спуск	0,0100	0,4516	0,4701
	0,0500	0,2616	0,2611
	0,2000	0,2488	0,2499
МНК (аналитически)	—	0,2487	0,2502



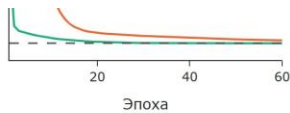


Рисунок 2. Тот же результат в Plotly: наведение показывает значения всех серий на эпохе

Реализация

Ядро эксперимента — функция одного прогона градиентного спуска (листинг 1).

Листинг 1. Градиентный спуск для парной линейной регрессии

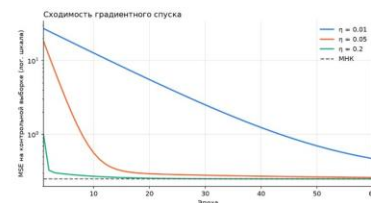
scripts/prepare.py

```
1 def gradient_descent(x, y, xv, yv, lr):
2     w = b = 0.0
3     train, val = [], []
4     for _ in range(EPOCHS):
5         err = w * x + b - y
6         w -= lr * 2 * np.mean(err * x)
7         b -= lr * 2 * np.mean(err)
8         train.append(mse(y, w * x + b))
9         val.append(mse(yv, w * xv + b))
10    return train, val
```

Строки 6–7 реализуют формулу (2).

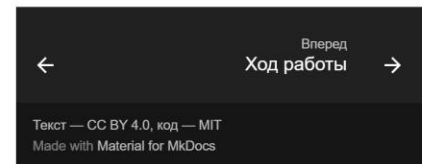
Интерпретация

При $\eta = 0,2$ градиентный спуск за 39 эпох достигает ошибки аналитического решения (см. таблицу 1), тогда как при $\eta = 0,01$ за 60 эпох ошибка остаётся почти вдвое выше. Малый шаг гарантирует устойчивость, но требует больше итераций¹. Вопросы публикации и цитирования таких результатов разобраны в разделе «Постоянные ссылки» отчёта по T5, а принципы FAIR — в работе⁵.



Литература

1. При $\eta > 1/\lambda_{\max}$, где $\lambda_{\max}$ — наибольшее собственное значение гессиана, итерации расходятся.
2. Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi:10.1126/science.1213847.
3. Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
4. John D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi:10.1109/MCSE.2007.55.
5. Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, and others. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi:10.1038/sdata.2016.18.



MkDocs + Material, мобильное разрешение 390 px.

Качество модели $y = wx + b$ оценивается среднеквадратичной ошибкой:

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2 \quad (1)$$

Параметры обновляются по правилу градиентного спуска со скоростью обучения η :

$$w \leftarrow w - \eta \frac{\partial \text{MSE}}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial \text{MSE}}{\partial b} \quad (2)$$

Минимизируемый функционал — формула (1), шаг обновления — формула (2). Подробное обоснование выбора метрики приведено в учебнике [Goodfellow et al., 2016].

Результаты

Таблица 1 Итоговая ошибка после 60 эпох (объединённая)

Метод	η	Итоговая MSE	
		обучающая	контр.
Градиентный спуск	0,0100	0,4516	0,41
	0,0500	0,2616	0,26
	0,2000	0,2488	0,24
МНК (аналитически)	—	0,2487	0,24

Кривые обучения на контрольной выборке

Кривые обучения на контрольной выборке показаны на Рисунок 1; график построен библиотекой Matplotlib [Hunter, 2007].

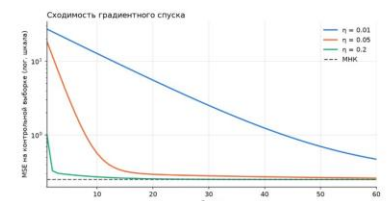
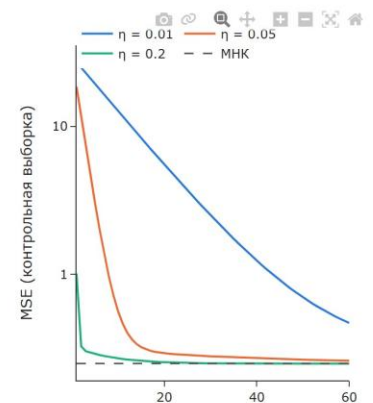


Рисунок 1 Сходимость градиентного спуска (статический PNG, Matplotlib)



Публикация результатов экспериментов

Стресс-тест научного контента (Sphinx + MyST)

Коммит 96dbd18 · сборка 2026-09-29 16:13 UTC · набор данных v1.0 (sha256 d4b31c5c5254)

Страница публикует результаты небольшого вычислительного эксперимента: обучение линейной регрессии градиентным спуском с разными скоростями обучения и сравнение с аналитическим решением методом наименьших квадратов (МНК). Графики и таблица формируются скриптом scripts/prepare.py из файла data/dataset.csv на этапе сборки, т. е. числа на странице всегда соответствуют текущей версии данных и кода [Peng, 2011].

Постановка эксперимента

эпоха

Тот же результат в Plotly: наведение показывает значения всех серий на эпохе (подпись не нумеруется: директива `figure` в Sphinx принимает только изображение).

Реализация

Ядро эксперимента — функция одного прогона градиентного спуска (Листинг 1).

Листинг 1. Градиентный спуск для парной линейной регрессии (scripts/prepare.py)

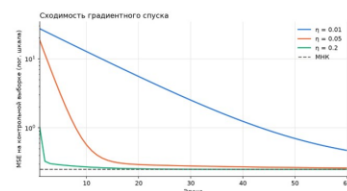
```
1 def gradient_descent(x, y, xv, yv, lr):
2     w = b = 0.0
3     train, val = [], []
4     for _ in range(EPOCHS):
5         err = w * x + b - y
6         w -= lr * 2 * np.mean(err * x)
7         b -= lr * 2 * np.mean(err)
8         train.append(mse(y, w * x + b))
9         val.append(mse(yv, w * xv + b))
10    return train, val
```

Строки 6–7 реализуют формулу (2).

Интерпретация

При $\eta = 0,2$ градиентный спуск за 39 эпох достигает ошибки аналитического решения (см. Таблица 1), тогда как при $\eta = 0,01$ за 60

эпох ошибка остаётся почти вдвое выше. Малый шаг гарантирует устойчивость, но требует больше итераций^[1]. Вопросы публикации и цитирования таких результатов разобраны в разделе «Постоянные ссылки» отчёта по T5, а принципы FAIR — в работе [Wilkinson et al, 2016].



Литература

- [1] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi:10.1126/science.1213847.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [3] John D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi:10.1109/MCSE.2007.55.

Sphinx + MyST, мобильное разрешение 390 px.

Мобильное разрешение 390 px

- [4] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, and others. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi:10.1038/sdata.2016.18.
- [1] При $\eta > 1/\lambda_{\max}$, где $\lambda_{\max}$ — наибольшее собственное значение гессиана, итерации расходятся.

Previous Home Next
Ход работы

Copyright © 2026, ExcaliBBur. Текст — CC BY 4.0, код — MIT
Made with Sphinx and @pradyunsg's Furo
Last updated on 2026-09-29

4. Что не удалось реализовать и какой ценой

Требование	MkDocs + Material	Sphinx + MyST
Нумерация формул	правка конфигурации JS: нумерует MathJax в браузере; номера существуют только внутри страницы, ссылка на формулу с другой страницы невозможна; без JavaScript номеров нет	из коробки; номера сквозные, есть и в HTML без JS
Объединённые ячейки	ручной HTML (Markdown-таблицы не умеют rowspan/colspan); в проекте HTML генерирует скрипт	вставка rST (grid table) внутри MyST: синтаксис псевдографики, требуется точное выравнивание столбцов — тоже генерируется скриптом
Подписи с автонумерацией	из коробки (pydownx.blocks.caption), но номер в тексте ссылки пишется вручную («рисунок 1»): аналога {numref} нет	из коробки: {numref} подставляет «Рисунок 1» сам
Подпись к интерактивному графику	из коробки (блок caption применим к любому содержимому)	не реализовано: {figure} принимает только изображение; без собственной директивы подпись не нумеруется
Библиография	плагин; цитаты оформляются как нумерованные сноски и смешиваются с обычными сносками в одном списке; стили CSL требуют Pandoc	из коробки после установки расширения; стиль «автор, год», сноски отдельно
Двухколоночный блок	правка CSS (6 строк): встроенный grid Material подбирает число колонок сам, фиксированные 2 колонки задаются вручную	из коробки (sphinx-design), адаптивность задаётся параметрами 1 1 2 2
Интерактивный график	iframe; тег <figure> в Material	iframe через {raw} html

Требование	MkDocs + Material	Sphinx + MyST
	сжимает iframe до 300 px — пришлось заменить на <div>	
Без внешних запросов	правка конфигурации: theme.font: false (иначе Google Fonts), локальный MathJax; остаётся запрос к GitHub API из- за repo_url	локальный MathJax (mathjax_path); Furo внешних запросов не делает

5. Вывод по P2

Оба генератора позволили собрать страницу целиком, но разной ценой. **Sphinx + MyST** покрывает научные элементы (нумерация и ссылки на формулы, рисунки, таблицы, листинги; библиография «автор, год») средствами самого генератора и не делает внешних запросов; за это платится вдвое большее время сборки и необходимость писать rST внутри Markdown для сложных таблиц. **MkDocs + Material** быстрее собирается и даёт более богатый интерфейс (поиск по мере ввода, тёмная тема, копирование кода), но нумерация формул переносится в браузер, ссылки на рисунки нумеруются вручную, а библиография смешивается со сносками.

Вес страницы определяется не генератором, а тяжёлыми библиотеками (Plotly.js и MathJax — 94 % веса). Их локальное размещение обязательно для работы без CDN и увеличивает объём сайта примерно на 6,8 МиБ, но не увеличивает вес страницы по сравнению с загрузкой тех же файлов с CDN.

Отладка

Ниже — ошибки, которые реально возникли при выполнении работы, в порядке появления. Для каждой указаны текст ошибки или симптом, гипотеза, способ проверки и решение.

1. Формулы в MkDocs выводятся сырым TeX

Симптом. На странице стресс-теста вместо формул виден текст `\(\hat y = wx + b\)`; при этом нумерованные формулы (1) и (2) отрисованы.

Гипотеза 1: MathJax не загрузился (404 или обращение к CDN). **Проверка:** вкладка Network — `assets/js/tex-svg.js` → 200; в консоли `MathJax.version === "3.2.2"`. Гипотеза отвергнута.

Гипотеза 2: MathJax не распознаёт разделители `\( \)`, а формулы (1), (2) отрисованы только потому, что `processEnvironments` находит `\begin{equation}` без разделителей. **Проверка:** в консоли `MathJax.config.tex.inlineMath` вернул `[["(", ")"]]` — обратных слешей нет. Байтовый дамп файла (`od -c mathjax-config.js`) показал `"\("` вместо `"\\("`: при записи файла через heredoc оболочки удвоенный слеш превратился в одинарный, а в строке JavaScript `"\("` — это просто `"(`.

Решение. Файл конфигурации переписан без участия оболочки, в нём `inlineMath`: `[["\\(", "\\)"]]`. Добавлена автоматическая проверка в `scripts/measure.py`: число элементов `.arithmatex` без отрисованного `mjx-container` должно быть 0.

2. mkdocs build --strict падает на ссылке на якорь

Текст ошибки:

```
WARNING - Doc file 'p2/stress-test.md' contains a link
'../report/t5.md#1-постоянные-ссылки-и-стабильность-url', but the doc
'report/t5.md' does not contain an anchor '#1-постоянные-ссылки-и-стабильность-url'.
Aborted with 1 warnings in strict mode!
```

Гипотеза: якорь не существует (раздел ещё не написан) или генерируется иначе, чем ожидается: стандартная функция `slugify` расширения `tos` удаляет все не-ASCII символы, и кириллический заголовок получает пустой `id`.

Проверка: после написания раздела T5 сборка со стандартным `slugify` выдала то же предупреждение; в HTML у заголовка «1. Постоянные ссылки и стабильность URL» оказался `id="1-url"` — кириллица вырезана. С `pymdownx.slugs.slugify(case=lower)` `id` стал `1-постоянные-ссылки-и-стабильность-url`, т. е. совпал с GitHub-стилем, который использует `MyST (myst_heading_anchors)`, и сборка прошла.

Решение. В `mkdocs.yml` задан `slugify: !!python/object/apply:pymdownx.slugs.slugify {kwds: {case: lower}}`; одна и та же ссылка теперь работает в обоих генераторах. Эта ошибка показывает, зачем в CI нужен `--strict`: без него сайт собрался бы с битой ссылкой.

3. Сниппет метки версии не раскрывается

Симптом. Вместо метки «Коммит ... · сборка ...» на странице выводится строка `--8<-- "build_info.md"`.

Гипотеза: `pydownx.snippets` обрабатывает только строку, которая **начинается** с `--8<--`, а в исходнике сниппет стоял внутри `<p class="build-info">...</p>` на одной строке.

Проверка: перенос `--8<--` в начало отдельной строки внутри `<div class="build-info" markdown>` — метка появилась.

Решение. Сниппет вынесен в отдельную строку внутри блока `md_in.html`.

4. Интерактивный график Plotly не растягивается на ширину колонки

Симптом. На странице MkDocs `iframe` с Plotly занимает узкую полосу по центру (≈ 480 px из 882 px колонки), хотя у него `width: 100%`.

Гипотеза 1: тег `<figure>` в теме Material имеет ширину по содержимому (`width: fit-content`), и 100 % ширины `iframe` считаются от неё. **Проверка:** обёртка `<figure markdown>` заменена на `<div>`. Скриншот не изменился — гипотеза в такой форме не подтвердилась.

Гипотеза 2: `<figure>` всё равно появляется: блок `/// figure-caption` расширения `pydownx.blocks.caption` оборачивает **предыдущий** элемент в `<figure>` и добавляет `<figcaption>`. **Проверка:** в консоли `document.querySelector('.plot-frame').closest('figure')` вернул элемент, `getComputedStyle(figure).width = 480px` при ширине контента 882 px. Гипотеза подтверждена.

Решение. Одно правило CSS: `.md-typeset figure:has(.plot-frame) { width: 100%; }`. Сопутствующий вывод: ширину элементов проверять замером в DOM, а не на глаз по скриншоту — первая «правка» лишь совпала с ожидаемым результатом в узком окне.

5. Healthcheck падает с UnicodeEncodeError на Windows

Текст ошибки:

```
File "...\\encodings\\cp1251.py", line 19, in encode
UnicodeEncodeError: 'charmap' codec can't encode character '→'
in position 16: character maps to <undefined>
```

Гипотеза: консоль Windows использует кодировку `cp1251`, в которой нет символа «→» из сообщений скрипта.

Проверка: `python -c "import sys; print(sys.stdout.encoding)"` → `cp1251`.

Решение. В начале `main()` вызывается `sys.stdout.reconfigure(encoding="utf-8")`. На раннере Ubuntu ошибка не проявилась бы — это пример различия локальной и CI-среды.

6. Healthcheck не находит русское слово в индексе поиска MkDocs

Симптом. FAIL MkDocs: `search_index.json` содержит слово 'градиентн...', хотя поиск на сайте это слово находит.

Гипотеза: MkDocs сериализует индекс с `ensure_ascii=True`, и кириллица хранится как гр..., поэтому поиск подстроки в сыром тексте файла ничего не находит.

Проверка: `head -c 300 public/search/search_index.json` — строки состоят из `\uXXXX`.

Решение. Файл разбирается `json.loads`, поиск идёт по полю `text` каждого документа.

7. `scripts/measure.py` не попадает в репозиторий

Симптом. После `git add -A` файл отсутствует в `git status`.

Гипотеза: шаблон `measure.py` в `.gitignore` (для старого черновика в корне) без ведущего / совпадает с файлом на любой глубине.

Проверка: после замены шаблона на `/measure.py` повторный `git add -A` добавил `scripts/measure.py` в индекс, а файл черновика в корне остался игнорируемым.

Решение. Шаблоны для файлов черновика привязаны к корню: `/measure.py`, `/mkdocs/`, `/sphinx/` и т. д.

8. Внешние запросы у MkDocs при «полностью локальном» сайте

Симптом. Тест в режиме «без CDN» показал 2 заблокированных запроса у MkDocs, хотя MathJax, Plotly и шрифты локальные.

Гипотеза: запросы делает тема, а не контент.

Проверка: в журнале запросов Playwright — два обращения к `api.github.com/repos/ExcaliBBur/static_site`; их инициирует компонент «source» темы Material, если задан `repo_url`.

Решение. Оставлено сознательно: запрос необязателен (без него не показывается только число звёзд), страница полностью работает. Для «строгой» автономности достаточно убрать `repo_url` из `mkdocs.yml`.

9. `mkdocs-bibtex` находит «цитату» внутри `inline`-кода

Текст ошибки (при добавлении отчёта в сайт MkDocs):

```
WARNING - Citing unknown reference key key
WARNING - Error formatting citation key into footnote format {key}
Aborted with 2 warnings in strict mode!
```

Гипотеза: в тексте отчёта по P2 синтаксис цитирования был показан как пример в обратных кавычках: открывающая скобка, «собака», слово `key`, закрывающая скобка. Плагин ищет цитаты регулярным выражением по исходному Markdown до его разбора и не знает, что это фрагмент кода.

Проверка: поиск `grep` по подстроке «скобка + собака» в `report/*.md` нашёл единственное вхождение в `report/p2.md`; ключа `key` в `.bib` нет. Первая попытка описать ошибку в этом разделе снова уронила сборку (Inline reference to unknown key key): при включённых `inline`-цитатах плагин реагирует даже на «собаку» перед словом без скобок.

Решение. Пример переписан словами. Ограничение плагина отмечено в сравнении P2: в Sphinx роль `{cite:p}` разбирается парсером и внутри кода не срабатывает.

Вывод

Рекомендуемый стек

Предметная область — вычислительные эксперименты по машинному обучению и численным методам: результаты — таблицы метрик, кривые обучения, формулы, фрагменты кода, ссылки на литературу.

Слой	Рекомендация	Обоснование (раздел отчёта)
Генератор	Sphinx + MyST-Parser (тема Furo), sphinxcontrib-bibtex, sphinx-design	нумерация и ссылки на формулы, рисунки, таблицы и листинги при сборке; библиография «автор, год»; ноль внешних запросов (P2, § 4)
Альтернатива для документации проекта	MkDocs + Material	в 2 раза быстрее сборка, лучше поиск и интерфейс; научные элементы требуют ручной работы (P2, § 4)
Графики	Matplotlib (PNG) по умолчанию; Plotly — только там, где интерактивность нужна для анализа	Plotly.js — 65 % веса страницы, 4,7 МиБ (P2, § 3.2)
Формулы	MathJax 3, локальная копия	работа без CDN проверена автоматически (P2, § 3.4)
Конвейер данных	скрипт prepare.py перед сборкой: данные → расчёт → графики/таблицы → метка версии	числа на странице не расходятся с данными; хеш данных виден читателю (T5, § 2)
CI/CD	GitHub Actions: upload-pages-artifact + deploy-pages, actions по SHA, healthcheck после деплоя	без долгоживущих токенов с правом записи в репозиторий
Хостинг	GitHub Pages — основной; Helios — зеркало в отечественной инфраструктуре	постоянство URL, HTTPS, свой домен недоступны на Helios (T5, § 8)
Публикация	Git-ter → GitHub Release → Zenodo (DOI), CITATION.cff, ORCID, мета-теги Highwire/DC/OG, отдельные лицензии CC BY 4.0 / MIT / CC0	FAIR и цитируемость обеспечивает инфраструктура, а не генератор (T5, § 6, 10)

Условия, при которых рекомендация меняется

Условие	Что выбрать вместо
Результаты — в основном Jupyter-ноутбуки, их нужно исполнять при сборке	Sphinx + MyST-NB (или Jupyter Book) — кэш исполнения ноутбуков
Нужны PDF/печатная версия того же текста	Sphinx (LaTeX-билдер) или Quarto; MkDocs PDF не делает
Нужна «публикация из коробки»: блок цитирования, ORCID, Highwire, DOI в метаданных	Quarto — если допустим не-Python инструмент в CI
Сайт — документация кода с частыми правками, важны поиск и навигация, формул мало	MkDocs + Material (+ mike для версий)
Сайт — блог/лента заметок об экспериментах	Pelican
Команда пишет только Markdown и не готова к синтаксису ролей и директив MyST	MkDocs + Material; научные элементы — ручной HTML и плагины
Нет доступа к GitHub, нужна только отечественная инфраструктура	та же сборка в GitVerse/SourceCraft CI; хостинг — объектное хранилище с собственным доменом, а не Helios
Результат должен цитироваться годами	обязательно Zenodo (DOI) + собственный домен; учебный хостинг не подходит

Условие	Что выбрать вместо
Нужна работа без JavaScript (печать, строгие требования доступности)	Sphinx (номера формул есть в HTML); MathJax заменить на серверный рендеринг формул в SVG

Приложение А. Тексты workflow

Полные тексты конфигураций CI/CD с комментариями. Все сторонние actions зафиксированы по SHA коммита; секреты берутся из хранилища GitHub и в репозитории отсутствуют.

.github/workflows/pages.yml

```
# Сборка обоих сайтов и публикация на GitHub Pages
# официальной связкой upload-pages-artifact + deploy-pages (без ветки gh-pages).
name: pages

on:
  push:
    branches: [main]
  pull_request:      # только сборка: секретов и прав на деплой нет
  workflow_dispatch:

# Минимальные права по умолчанию; расширяются только в job deploy.
permissions:
  contents: read

# Одновременно выполняется одна публикация; незавершённую не прерываем,
# чтобы сайт не остался в промежуточном состоянии.
concurrency:
  group: pages
  cancel-in-progress: false

env:
  SITE_URL: https://excalibbur.github.io/static_site/

jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      # Сторонние actions зафиксированы по SHA коммита (тег можно переписать).
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1

      - uses: actions/setup-python@5fda3b95a4ea91299a34e894583c3862153e4b97 # v7.0.0
        with:
          python-version: "3.13.3"
          cache: pip
          cache-dependency-path: requirements.txt

      - name: Install dependencies (pinned)
        run: pip install -r requirements.txt

      - name: Build MkDocs + Sphinx (strict)
        run: python scripts/build_all.py

      - name: Healthcheck of the build output
        run: |
          python -m http.server 8000 --directory public >/dev/null 2>&1 &
          sleep 2
          python scripts/check_site.py http://127.0.0.1:8000/ --commit "${GITHUB_SHA::7}"

      - uses: actions/upload-pages-artifact@fc324d3547104276b827a68afc52ff2a11cc49c9 # v5.0.0
        with:
          path: public

  deploy:
    # Публикация только из main; для PR и других веток — только build.
    if: github.event_name != 'pull_request' && github.ref == 'refs/heads/main'
    needs: build
    runs-on: ubuntu-24.04
    permissions:
      pages: write      # развернуть артефакт на Pages
      id-token: write   # OIDC-токен для подтверждения происхождения артефакта
```

```

environment:
  name: github-pages
  url: ${ steps.deployment.outputs.page_url }}
steps:
  - id: deployment
    uses: actions/deploy-pages@368f82528645a54fb793d4d04e342629a3f51346 # v5.0.1

  - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
    with:
      sparse-checkout: scripts

  - name: Healthcheck of the published site
    run: python3 scripts/check_site.py "$SITE_URL" --commit "${GITHUB_SHA:7}" --wait-commit 180

```

.github/workflows/helios.yml

```

# Выкладка того же сайта на Helios ИТМО (rsync по SSH с отдельным deploy-ключом).
#
# Настройки репозитория (Settings → Secrets and variables → Actions):
# secrets.HELIOS_SSH_KEY      - приватный deploy-ключ (ed25519, без пароля),
#                               публичная часть - в ~/.ssh/authorized_keys на Helios:
# restrict,command="/usr/local/sbin/rsync /home/studs/s335989/public_html/p2/" ssh-ed25519 ...
#                               ключ позволяет только rsync в каталог сайта (путь ".")
# secrets.HELIOS_KNOWN_HOSTS  - вывод `ssh-keyscan -p 2222 se.ifmo.ru`,
#                               сверенный с отпечатком, полученным по доверенному каналу
# vars.HELIOS_USER            - логин на Helios (sXXXXXX)
name: helios

on:
  push:
    branches: [main]
    workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: helios
  cancel-in-progress: false

env:
  HELIOS_HOST: se.ifmo.ru
  HELIOS_PORT: "2222"

jobs:
  deploy:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-24.04
    env:
      # Сайт лежит в подкаталоге ~/логин/p2/. Обычные страницы MkDocs используют
      # относительные ссылки, но 404.html, rel=canonical и sitemap.xml строятся
      # от site_url: с адресом GitHub Pages 404-страница на Helios ломается.
      SITE_URL: https://se.ifmo.ru/~${ vars.HELIOS_USER }}/p2/
    environment:
      name: helios
      url: https://se.ifmo.ru/~${ vars.HELIOS_USER }}/p2/
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1

      - uses: actions/setup-python@5fda3b95a4ea91299a34e894583c3862153e4b97 # v7.0.0
        with:
          python-version: "3.13.3"
          cache: pip
          cache-dependency-path: requirements.txt

      - run: pip install -r requirements.txt

      - name: Build MkDocs + Sphinx (strict)
        run: python scripts/build_all.py

```

```

- name: Configure SSH (deploy key + pinned host key)
  env:
    SSH_KEY: ${ secrets.HELIOS_SSH_KEY }
    KNOWN_HOSTS: ${ secrets.HELIOS_KNOWN_HOSTS }
  run: |
    install -d -m 700 ~/.ssh
    install -m 600 /dev/null ~/.ssh/helios
    printf '%s\n' "$SSH_KEY" > ~/.ssh/helios
    # Ключ хоста берётся из секрета, а не из ssh-keyscan в момент деплоя:
    # так подмена сервера (MITM) приведёт к ошибке, а не к утечке сайта.
    printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts
    chmod 600 ~/.ssh/known_hosts

- name: Deploy with rsync
  run: |
    rsync -rlz --checksum --delete-after --delay-updates \
      --chmod=D755,F644 \
      -e "ssh -p $HELIOS_PORT -i ~/.ssh/helios -o StrictHostKeyChecking=yes -o IdentitiesOnly=yes"
    \
      public/ "${ vars.HELIOS_USER }}@$HELIOS_HOST:./"

- name: Healthcheck
  run: python scripts/check_site.py "https://se.ifmo.ru/~${ vars.HELIOS_USER }}/p2/" --commit
    "${GITHUB_SHA:~7}"

- name: Remove key
  if: always()
  run: rm -f ~/.ssh/helios

```

.github/workflows/vps.yml

```

# Выкладка на собственный VPS (excalibur.ru, Ubuntu 24.04, Nginx Proxy Manager).
#
# Файлы сайта лежат в /root/npm/data/static_site и отдаются NPM по адресу
# https://excalibur.ru/static_site/ (location в /data/nginx/custom/server_proxy.conf).
#
# Deploy-ключ в authorized_keys ограничен принудительной командой rrsync:
# restrict,command="/usr/bin/rrsync /root/npm/data/static_site/" ssh-ed25519 ...
# Скомпрометированный ключ позволяет только перезаписать файлы сайта:
# шелла, форвардинга портов и доступа к другим каталогам нет.
#
# secrets.VPS_SSH_KEY      – приватный deploy-ключ
# secrets.VPS_KNOWN_HOSTS  – ED25519-ключ хоста excalibur.ru (сверен с доверенным)
name: vps

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: vps
  cancel-in-progress: false

jobs:
  deploy:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-24.04
    env:
      SITE_URL: https://excalibur.ru/static_site/
    environment:
      name: vps
      url: https://excalibur.ru/static_site/

```

```

steps:
- uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1

- uses: actions/setup-python@5fda3b95a4ea91299a34e894583c3862153e4b97 # v7.0.0
  with:
    python-version: "3.13.3"
    cache: pip
    cache-dependency-path: requirements.txt

- run: pip install -r requirements.txt

- name: Build MkDocs + Sphinx (strict)
  run: python scripts/build_all.py

- name: Configure SSH (deploy key + pinned host key)
  env:
    SSH_KEY: ${ secrets.VPS_SSH_KEY }
    KNOWN_HOSTS: ${ secrets.VPS_KNOWN_HOSTS }
  run: |
    install -d -m 700 ~/.ssh
    install -m 600 /dev/null ~/.ssh/vps
    printf '%s\n' "$SSH_KEY" > ~/.ssh/vps
    printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts

- name: Deploy with rsync (rrsync on the server side)
  run: |
    # Путь назначения "/" – корень, разрешённый rrsync на сервере.
    rsync -rlz --checksum --delete-after --delay-updates \
      --chmod=D755,F644 \
      -e "ssh -i ~/.ssh/vps -o StrictHostKeyChecking=yes -o IdentitiesOnly=yes" \
      public/ root@excalibbur.ru:./

- name: Healthcheck
  run: python scripts/check_site.py "$SITE_URL" --commit "${GITHUB_SHA::7}"

- name: Remove key
  if: always()
  run: rm -f ~/.ssh/vps

```